forge-plugin

для владельца плагина и людей со стороны · 2026-09-06 · версия проекта 7.8.0

Гайд по проекту v1.0

плагин работает у владельца, версия 7.8.0; фаза 5 переезжает на гайд

00 Где мы сейчас — за 30 секунд

Плагин работает у владельца — установлен в его Claude Code, опубликован на GitHub; проверки служебных скриптов — 122 из 122 прошли.

Петля замкнута до слияния — задача → план → критика → код → слияние проходят на этом же проекте; фаза 5 (гайд) ещё в ветке.

Память сама, тесты руками — память проекта уезжает в git после сессии и слияния; автопроверки при изменениях нет, тесты запускают вручную.

До версии 7.8.0 — влить ветку гайда в основную и выложить 7.8.0 на GitHub; ответить на O2 и остальные открытые вопросы.

🔥 Решения, без которых стоим — нужны на этой неделе

- **O1** — Сколько у проекта «витрин»: только гайд — или ещё карта задач на GitHub?

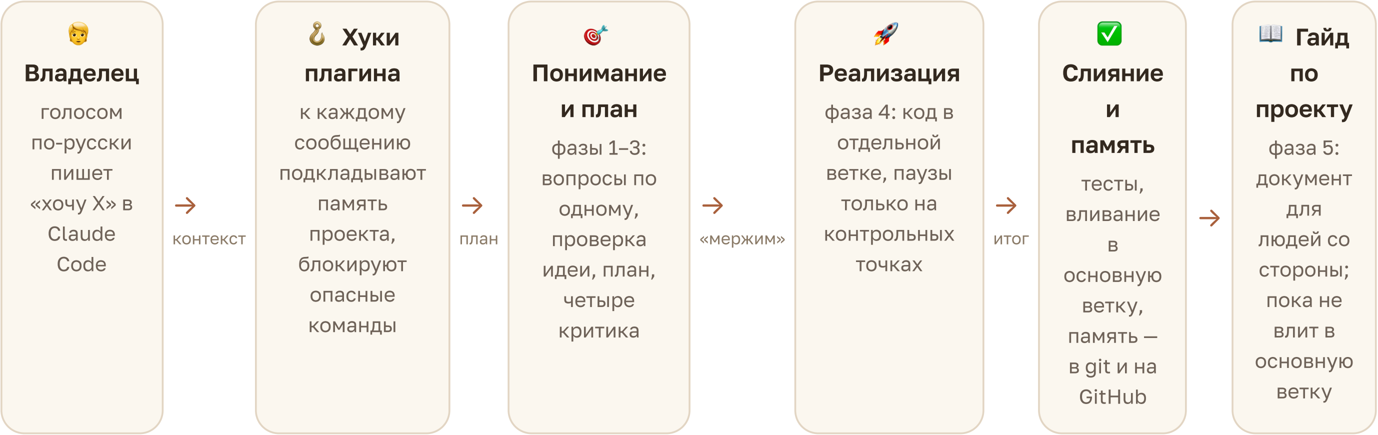
Как работать с документом. Каждое решение и риск имеет код: **P*** — процесс, **A*** — рамка: для кого плагин и как он говорит, **B*** — механика конвейера, **C*** — память проекта и GitHub, **D*** — качество и проверки, **O*** — открытые вопросы, **R*** — риски. Метка 🔥 — нужно на этой неделе, без этого стоим. Метка «дефолт» — наша рекомендация: нет возражений — действуем (см. P1). Отвечайте списком кодов — так / не так / как переделать:

```
A2 — ок.  B2 — переделать: …и как именно.  D2 — обсудить.
```

Ответы — владельцу проекта в любом виде: списком кодов, голосом, в переписке; он вносит их в гайд, и следующая версия выйдет с обновлёнными статусами

01 Что это такое

forge-plugin — надстройка (плагин) для Claude Code, помощника-программиста от Anthropic. Она ведёт владельца, который не пишет код, от фразы «хочу X» до готовой и влитой в проект правки по семи обязательным шагам, а между сессиями хранит память проекта и сама сохраняет её в git. Работает у самого владельца: плагин установлен в его Claude Code и опубликован на GitHub; есть ли другие пользователи — не проверяли.



👱 **Владелец** — голосом по-русски пишет «хочу X» в Claude Code

→ контекст

🪝 **Хуки плагина** — к каждому сообщению подкладывают память проекта, блокируют опасные команды

→ план

🎯 **Понимание и план** — фазы 1–3: вопросы по одному, проверка идеи, план, четыре критика

→ «мержим»

🚀 **Реализация** — фаза 4: код в отдельной ветке, паузы только на контрольных точках

→ итог

✅ **Слияние и память** — тесты, вливание в основную ветку, память — в git и на GitHub

→

📖 **Гайд по проекту** — фаза 5: документ для людей со стороны; пока не влит в основную ветку

После каждого шага Клод показывает сводку и ждёт «ОК» или «стоп»; вопросы задаёт по одному. Опасные команды останавливаются до выполнения, память проекта подкладывается автоматически, а при старте сессии плагин напоминает, если память давно не сохранялась.

02 Из чего состоит

🧭 Конвейер из семи шагов → B1, B3, P2, P3, P5

Вы пишете по-русски «хочу…», «почини…», а плагин ведёт Клода по фиксированным шагам: понять задачу → проверить идею → составить план → разнести план четырьмя критиками → сделать с остановками на контрольных точках → собрать гайд. На этом проекте цепочка пройдена вживую: есть файл задачи, план и разбор критики по текущей задаче.

🧠 Память проекта между сессиями

Цель, стадия, решения, тупики и журнал хранятся в папке памяти, и при каждом сообщении Клоду подкладывается короткая выжимка (полная — только когда что-то изменилось или раз в 15 сообщений), чтобы он не начинал с нуля. Проверено тестами: 12 из 12 прошли.

💾 Сохранение памяти в git → C1

Память уезжает в репозиторий отдельными записями по слову «сохрани» и сама после итога сессии и слияния; при старте сессии плагин напомнит, если несохранённое старше суток. Тесты: 14 из 14, включая случай, когда отправка на сервер не удалась.

🛡️ Страховка от опасных команд → D1

До выполнения любой команды в терминале плагин проверяет её и блокирует разрушительные (стереть диск, перезаписать основную ветку, удалить ключи); свои правила добавляются словами «чтобы это не повторялось». Тесты: 26 + 8 прошли.

📖 Гайд по проекту (шаг 5) → B2, B6, C2

Один версионный документ для человека со стороны: суть, устройство, решения с кодами, риски, план; ответы кодами в чат меняют его без пересборки. Сборщик написан и проверен 56 тестами; этот документ — первая собранная версия, работа ещё не влита в основную ветку.

🔗 Связь задач с GitHub → C3, O1

Обещано: каждая задача — карточка (Issue) на GitHub, шаги — подкарточки, закреплённая карта целей и живая шапка в README. На маке владельца цепочка не работает: команда включения падает, привязка к целям молча не срабатывает, шапка README застыла на 26 мая.

🧰 Навыки-помощники под ситуацию → A1, P4

Помимо конвейера — около 28 навыков, которые Клод включает по вашим словам: разобраться «почему не работает» и починить, проверить безопасность, спроектировать интерфейс (поиск по базе дизайн-правил проверен запуском), выложить на сервер, ревью кода, закрытие ветки. Что они включаются по фразе, проверено лишь для 10 из 35.

🖥️ Удобства в рабочем окне → A2

Краткий стиль ответов без воды (включается вместе с плагином), звуки на «готово» и «нужно разрешение» (7 + 7 записей подключены), полоска внизу окна с текущим шагом и веткой. Полоска у владельца не включена.

⚒ Меню команд в строке состояния

Маленькая иконка на маке: клик по пункту копирует команду плагина, чтобы не набирать её. Версия для мака знает все 7 шагов (в рабочей ветке пятый пункт уже переименован в гайд); версия для Linux предлагает команды, которых в плагине давно нет.

🧪 Проверка самого плагина → D2, O3

Шесть наборов тестов на служебные скрипты (122 проверки, все прошли при ручном запуске) и заготовка системы оценки качества по реальным диалогам. Автозапуска нет, старые тесты навыков на маке не стартуют, а система оценки пуста.

03 Главный путь по шагам

1. **«Хочу Х».** владелец пишет любую просьбу — «хочу», «нужно», «добавь», «сделай». Клод сам направляет её в фазу 1; жалоба на поломку уходит в разбор проблемы, «почини» — в отладку.

2. **Понимание задачи.** Клод задаёт только смысловые вопросы (что, кому, зачем) по одному; техническое ищет сам. Итог — файл задачи с разделами «Задача» и «Критерий готовности». После «да, оно» Клод сам переходит к проверке идеи.

3. **Проверка идеи.** Клод пересказывает идею своими словами и спорит с ней фактами проекта: та ли проблема, нет ли пути проще, что сломает, стоит ли усилий. На каждую слабость — конкретная альтернатива; здравую идею не оспаривает ради спора.

4. **План с паузами.** план из маленьких шагов с контрольными точками на смысловых границах и разделом о том, что увидит владелец. Если задача упирается в большое отдельное дело, Клод останавливается и предлагает решить его в отдельной сессии.

5. **Четыре критика.** четыре независимых помощника (Скептик, Прагматик, Архитектор, Адвокат пользователя) одновременно ищут дыры в плане; в сводку попадают только находки с уверенностью не ниже 80 %. Если чат уже тяжёлый, Клод рекомендует новый чат.

6. **Реализация в ветке.** Клод молча заводит отдельную ветку под задачу, выполняет шаги, тяжёлое отдаёт помощникам, останавливается только на контрольных точках и проверяет их реальным запуском. Пробел в плане — стоп и вопрос, не импровизация. В конце — по каждому пункту критерия доказательство или честный ❌ и вопрос, вливать ли в основной проект.

7. **Слияние.** по слову «мержим» Клод гоняет тесты (не прошли — не сливает), показывает, что сохранит, вливает ветку в основную и удаляет её; при конфликте останавливается и объясняет простыми словами. Затем отмечает задачу в гайде как сделанную и сохраняет память проекта в git с отправкой на GitHub.

8. **Гайд по проекту.** по слову «собери гайд» Клод заводит новую версию, шесть помощников проходят по коду и памяти, сборщик собирает HTML и PDF и открывает документ; владелец отвечает на решения кодами («B1 ок», «D1 — переделать: …»). Первая версия — этот документ; в основную ветку ещё не влито.

04 Кто и что делает

Роль	Что видит и делает
Владелец проекта	не пишет код, говорит по-русски голосом: «хочу…», «почини…», «собери гайд»; отвечает «ок» или кодами вроде «A2 ок». Видит вопросы по одному, план с контрольными точками, результат в браузере; технических вопросов ему не задают.
Клод внутри Claude Code	исполнитель: при старте получает памятку плагина и память проекта, по словам владельца выбирает навык, ведёт шаги конвейера, тяжёлую работу отдаёт помощникам, записывает решения и тупики в память.
Помощники-субагенты	отдельные экземпляры Клода, запускаемые параллельно: четыре критика плана, шесть аналитиков и картографов при сборке гайда, проверяющие код перед слиянием. Владелец их не видит — получает только сведённый итог.
Читатель гайда со стороны	партнёр, заказчик или сам владелец через месяц: открывает последнюю версию гайда (HTML или PDF), понимает проект целиком и отвечает на решения по их кодам.
Сторонний установщик плагина	ставит плагин из каталога двумя командами по описанию плагина. Если пойдёт по большому руководству — попадёт на чужой адрес и устаревшие команды версии 5.0; что нужно на компьютере, нигде не сказано.
Разработчик плагина	сам владелец вместе с Клодом: правит тексты навыков и скрипты, гоняет тесты руками по одному файлу, выкладывает новую версию на GitHub.

05 Решения

Каждое — как заложено сейчас и почему. Ждём по каждому: ок / переделать / обсудить.

P — Процесс

P1 **Правило дефолта** дефолт

По каждому пункту с меткой «дефолт» есть наша рекомендация. Если возражений нет — действуем по ней и не стоим; передумать можно позже.

Почему: Двигает проект вперёд без бесконечных согласований, и всегда видно, что и когда решили.

P2 **Одна задача — одна ветка, слияние по слову «мержим»** уже работает

Каждая задача живёт в своей ветке, которую плагин заводит сам при входе в реализацию; по слову «мержим» он сохраняет несохранённое, вливает работу в основную ветку и убирает рабочую.

Почему: Основная ветка всегда остаётся рабочей, чужая незавершённая работа не смешивается с новой.

P3 **Код — только после утверждённой задачи и раскритикованного плана** уже работает

Реализация не начинается без файла задачи с критерием готовности и без плана, прошедшего критику; «простых» задач, для которых фазу понимания можно пропустить, не бывает.

Почему: Половина прошлых провалов начиналась с задачи, не понятой до планирования, а без критики дыры плана всплывают дорого — уже в коде.

P4 **Технику Клод решает сам, владельцу — только смысл, по одному вопросу** уже работает

Вопросы про стек, библиотеки и устройство кода Клод закрывает сам чтением кода и документации; владельцу задаются только вопросы «что, для кого, зачем», по одному за раз; вместо списка вариантов даётся рекомендация.

Почему: Владелец — не программист: технические вопросы для него — ощущение, что его заваливают непонятным; а смысловые вопросы может решить только он, без них задача поставлена криво.

P5 **После «ОК» следующая фаза стартует сама; гайд — только по слову** принято

Короткое «ОК», «го», «дальше» после фазы запускает следующую без отдельной команды; «стоп», «пауза», «погоди» останавливают; если чат уже тяжёлый, между критикой и реализацией Клод предлагает новый чат. Гайд (фаза 5) в авто-цепочку не входит — собирается по слову, а после слияния обновляется сам.

Почему: Одним «ок» можно прокатить весь конвейер, не набирая команды, и при этом нельзя случайно выскочить из критики в реализацию с дырявым планом.

A — Рамка: для кого плагин и как он говорит

A1 **Пользователь — не программист с голосовым вводом по-русски** уже работает

Все навыки срабатывают на русские слова («хочу», «почини», «собери гайд»), объясняют без жаргона, опечатки и оборванные фразы считают нормой и не переспрашивают «вы имели в виду X?».

Почему: Владелец не пишет код и вводит голосом; скорость понимания важнее полноты.

A2 **Отвечать коротко — через встроенный стиль ответов Claude Code, а не через самодельные хуки** уже работает

Правила краткости живут во встроенном стиле ответов «Forge Concise», который включается сам вместе с плагином; если в Claude Code есть штатный механизм (стиль, полоска состояния, проверка перед командой) — используется он, а не самодельный хук.

Почему: Раньше правила стиля вставлялись в каждый запрос через хук и тратили объём чата; встроенный стиль включается один раз — меньше расходов, проще отладка.

B — Механика конвейера

B1 **Семь фаз конвейера, у каждой свой вход, выход и след в памяти** уже работает

Направление → понимание задачи → разбор идеи → план → критика → реализация → гайд; у каждой фазы своя команда и свой файл в памяти проекта.

Почему: Старая схема смешивала понимание задачи и планирование и не имела шага критики; разведение фаз даёт чёткий вход, выход и след на каждом шаге.

B2 **Фаза 5 — живой версионный гайд по проекту вместо отчёта «Что дальше»** принято

Один документ для читающего со стороны (суть, устройство, роли, решения с кодами, риски, план), версии с номерами и PDF на виду; каждая сборка дополняет прошлую версию, ответы владельца кодами меняют реестр; после слияния задачи обновляется сам. Код готов в рабочей ветке, в основную ветку и в версию 7.8.0 ещё не выложен.

Почему: Два документа «что чинить / что решать» и «где мы» для одного человека — путаница; нужен один живой документ, из которого человек со стороны понимает проект целиком, а история версий не пишется с нуля.

B3 План рвут четыре критика; в план попадает только уверенное (≥ 80 %) уже работает

Скептик, Прагматик, Архитектор и Адвокат пользователя разбирают план одновременно; находки с уверенностью ниже 80 % отбрасываются; при фатальной дыре реализация не начинается — план возвращается на доработку.

Почему: Планировщик влюблён в свой план — свежие глаза ловят дыры дешевле, чем разбор в коде; порог уверенности отсекает шум.

B4

Приёмы поиска неизвестного: слепые зоны, «что увидит владелец», макет до кода, журнал отклонений

уже работает

В понимании задачи — проход по слепым зонам, если область для владельца новая; в плане — первым блоком «что увидит владелец» и обязательный макет экрана до кода для задач с интерфейсом; в реализации — журнал отклонений от плана. Проверочный опрос перед слиянием отвергнут владельцем.

Почему: У человека без опыта в коде максимум «неизвестных неизвестных»; реакция на картинку работает у него лучше текста; отклонения иначе растворяются молча; опрос при каждом слиянии раздражал бы.

B5

Навигатор расписывает все направления и рекомендует, но выбор оставляет владельцу; отдельной дорожной карты не ведёт

принято

Фаза 0 показывает карту проекта со статусами частей, перечисляет все направления к цели, рекомендует движение по умолчанию и ждёт реакции; отдельный документ «для глаз» навигатор не пишет — витрина одна, гайд. По-настоящему навигатор на этом проекте ещё не запускали (см. О4).

Почему: Владелец слаб в придумывании стратегии, но финальный выбор за ним; два документа «где мы» для одного человека — путаница.

B6

Вид и место гайда: оформление эталона, папка на виду, PDF через Chrome, коды P/O/R плюс латинские группы

дефолт

Предлагаем закрепить: тёплая палитра и шрифты эталона, печать на A4, ширина на экране 960 px; видимые версии лежат в открытой папке документов, а не в скрытой папке памяти; PDF собирается через установленный Chrome, без Chrome — только HTML; группы кодов P, O, R всегда, остальные буквы подбираются под проект.

Почему: Эти пункты Клод принял сам с пометкой «владелец может отменить»; владелец уже реагировал на живой макет (попросил ограничить ширину) и не возражал против остального; менять формат после первой собранной версии дороже, чем подтвердить сейчас.

C — Память проекта и GitHub

`C1` Память проекта — в git и на GitHub; секреты — только названия записей в хранилище паролей

уже работает

Задачи, планы, решения, журнал и данные гайда сохраняются в репозиторий и отправляются на GitHub сами (итог сессии, слияние, слово «сохрани»); служебный мусор отсечён; при несохранённом дольше суток плагин напоминает; без удалённой копии предлагает приватный репозиторий только с явного «да»; пароли и ключи в память не пишутся.

Почему: Память должна переживать смерть диска и сервера во всех проектах; локальная папка — единая точка отказа; история изменений памяти — бесплатный бонус.

`C2` Данные гайда — JSON, не YAML; HTML собирает сборщик, Клод HTML руками не пишет

уже работает

Данные фазы 5 хранятся в файлах версий в памяти проекта, страницу и PDF собирает скрипт на стандартной библиотеке Python; после слияния и ответов владельца документ пересобирается механически.

Почему: На маке владельца нет библиотеки YAML, а JSON есть «из коробки»; данные плюс шаблон не дают сломанной вёрстки, обновление после слияния становится механическим шагом.

`C3` Связь с GitHub — через утилиту gh из скрипта, вызовы из навыков фаз, без досок Projects

принято

Задачи → карточки (Issues), шаги плана → подкарточки, карта целей → закреплённая карточка и шапка README; всё делает один скрипт через утилиту gh, вызываемый из навыков после завершения фазы, а не хуком на каждое действие; доска Projects не используется. Решение записано и воплощено в коде, но на маке владельца цепочка сейчас не работает (R1–R3) — судьба в O1.

Почему: Хук на каждое действие срабатывал бы много раз за фазу; доска дублирует закреплённую карточку и требует лишних прав, которых у большинства пользователей нет.

D — Качество и проверки

`D1` Опасные команды и правила владельца проверяются до выполнения

уже работает

Хук перед выполнением команды отсекает удаление диска, перезапись основной ветки и подобное; второй хук перед правкой файла сверяет действие с правилами владельца, зафиксированными по слову «чтобы не повторялось».

Почему: Разрушительные команды должны останавливаться раньше, чем дойдут до терминала; правила «чтобы не повторялось» действуют в новых сессиях без напоминаний.

`D2` Логика хуков и сборщика — через тесты; «готово» — после запуска

уже работает

Логика хуков, сборщика гайда и сохранения памяти пишется через тесты (сначала падающий тест, потом код); вёрстка и настройки тестами не обкладываются; заявить «работает» можно только после реального запуска с цитатой вывода.

Почему: Тесты без изоляции однажды испачкали настоящий репозиторий и отправили мусорные записи на GitHub; без запуска «должно работать» — не доказательство.

O — Открытые вопросы

01 🔥 ЭТА НЕДЕЛЯ **Сколько у проекта «витрин»: только гайд — или ещё карта задач на GitHub?**

открыто

Оставить карту на GitHub (карточки под задачи, закреплённая карта целей, шапка README) — тогда чинить включение на маке (R1), привязку задач к целям (R2) и шапку (R3), а потом покрыть скрипт тестами (R10); или выключить синхронизацию и оставить гайд единственным документом «где мы», как записано в решении о фазе 5.

02 **Плагин — личный инструмент владельца или продукт для других, кто не пишет код?** принято

Личный инструмент владельца. Друг пользуется бесплатно как есть — под него ничего не делаем (Антон, 2026-10-23)

~~было: Если продукт для других — руководство, страница на GitHub, установка «с нуля», проверка всех навыков по фразам становятся обязательными (R8, R9, R13, R24), а личное обращение к владельцу по имени из стиля ответов убирается; если личный инструмент — эти находки уходят в «потом», а обещание «для любого, кто пишет код через Claude Code» снимается из описания.~~

Ваш ответ 2026-10-23: Личный инструмент владельца. Друг пользуется бесплатно как есть — под него ничего не делаем (Антон, 2026-10-23)

03 **Оценка качества плагина по реальным сессиям: собирать корпус сейчас или отложить?** открыто

Заготовка есть: критерии по каждой фазе и таблица разбора ошибок, но корпус из ~116 сессий владельца не собран и ни один прогон не делался. Варианты: собрать записи сессий, разметить провалы и прогнать (несколько сессий работы) — или честно пометить папку отложенной идеей и не считать её работающей.

04 **Запускать ли навигатор направления по-настоящему?** открыто

Файл стратегии пуст: карта, направления, запас идей — все списки пустые, петля «разговор → память → задачи → снова память» ни разу не замкнулась. Варианты: провести первый разговор с навигатором и записать направления в память — или снять это обещание из описания плагина.

05 **Судьба задачи «отделить проверенные факты от догадок» (заведена 7 июля)** открыто

Задача сформулирована владельцем 7 июля и заведена на GitHub, плана к ней нет два месяца. Варианты: взять следующей после гайда, положить в запас идей или закрыть.

06 **Судьба черновиков и посторонних файлов в корне репозитория** открыто

Папка черновиков (27 файлов, из них 11 версий одного прототипа) и четыре посторонних файла в корне не менялись с апреля и ниоткуда не вызываются. Варианты: убрать в архив (ничего не теряется), удалить или оставить как есть.

06 Карта рисков

Отсортировано по срочности. Меры — наши предложения, отвечайте по кодам R.

🔴 Критичные — закрыть в первую очередь

R16 Скрипты плагина вызываются по адресу, который в сессии пуст — сохранение памяти и обновление гайда держатся на догадке Клода

Синхронизация с GitHub, сохранение памяти в git, пересборка гайда после слияния, связь карточки с задачей — все зовут скрипт по переменной, которая в сессии оказалась пустой. Клод либо тратит ход на поиск файла, либо пропускает шаг, а владелец слышит «готово». Пока выручает то, что Клод ищет папку сам; фаза 5 целиком зависит от этих вызовов.

Предлагаем: Привести все 37 вызовов в навыках к форме, которую Claude Code подставляет в текст (со скобками, как уже сделано в хуках), добавить в тесты проверку, чтобы старая форма не возвращалась, и запасной путь к папке плагина на случай пустой переменной.

R1 Команда «включи GitHub» на маке не срабатывает (→ O1)

Владелец отвечает «да» на предложение включить синхронизацию — плагин выдаёт служебную ошибку, флаг не ставится, и вся связь задач с GitHub дальше тихо не работает; выглядит как «просто выключено». Проверено 6 сентября живым запуском на маке.

Предлагаем: Переписать запись флага в память проекта способом, который одинаково работает на маке и Linux, и добавить тест на маковский вариант команды правки файла (sed).

R2 Задачи молча не привязываются к целям на GitHub (→ O1)

Задача заводится без цели, ошибка подавлена. Карта целей на GitHub показывает шесть целей с нулём задач в каждой — а по ней владелец и смотрит, что куда идёт. Известно с 10 июля и записано в уроки проекта, код не исправлен.

Предлагаем: Передавать GitHub название цели вместо номера (или превращать номер в название запросом) и поправить две инструкции навыков, которые учат передавать номер.

R3 Шапка «над чем идёт работа» в README не обновляется с мая (→ O1)

Заявлено как работающая возможность, но на любой машине без этой библиотеки скрипт тихо выходит «успешно». На маке владельца Python (язык скриптов плагина) в сессиях Клода запускается из окружения трея, где этой библиотеки нет. Шапка застыла на 26 мая и говорит о задаче, закрытой три месяца назад — посторонний читатель видит неверное состояние проекта.

Предлагаем: Читать два поля из памяти простым разбором строк, без дополнительной библиотеки для чтения памяти проекта (YAML) — как это уже делают другие скрипты плагина; если библиотека всё же нужна — заметное предупреждение владельцу вместо тихого выхода.

~~R22 Гайд ещё не влит в основную ветку~~

→ уже работает

🟡 Средние — держать в поле зрения

R24 Плагин не говорит, что ему нужно на компьютере, и молча отключает части, когда чего-то нет

Установка обещана двумя командами, но плагин зависит от четырёх внешних программ. Когда чего-то нет, часть отключается молча: владелец видит «GitHub не обновляется» или «PDF не появился» и не понимает, что это не поломка. Хуже всего с защитой от опасных команд: при сломанном Python она сознательно пропускает всё, а предупреждение уходит только в отладочные логи, которые никто не читает.

Предлагаем: Раздел «что должно быть на компьютере» в описании плагина (Python — язык скриптов плагина; библиотека YAML — для связи с GitHub; утилита gh — для GitHub; Chrome — для PDF) и одна проверка при старте сессии: если Python не запускается или при включённом GitHub чего-то нет — сказать во вводной строке простыми словами, чего не хватает и как поставить.

R11 Тесты запускаются только если про них вспомнить руками

Сейчас все 122 проверки зелёные, но узнать, что очередная правка что-то сломала, можно, только вспомнив запустить их. Владелец правит через Клода: сломанный хук (защита от опасных команд, загрузка памяти, напоминание о гайде) доедет до установленного плагина незамеченным.

Предлагаем: Один запускатор всех быстрых тестов и автозапуск на GitHub при каждом изменении: шесть наборов проверок служебных скриптов (без сети и без вызова Клода) плюс проверка сборки; медленные тесты с вызовом Клода оставить ручными.

R15 Часть тестов невозможно запустить на маке — не хватает системной утилиты

Владелец, решив «прогнать все тесты», по инструкции попадёт в набор, который на маке падает на первой строке, и решит, что плагин сломан. Записано в журнал как «следующее» ещё 6 июля.

Предлагаем: Либо доделать запасной вариант без утилиты timeout в старом наборе тестов (в наборе проверки навыков по фразам уже сделано), либо объявить три старых набора архивом «только Linux» в описании проекта и убрать их из инструкции по тестам.

R10 Самый большой скрипт плагина трогает настоящие задачи на GitHub — и не проверен ничем (→ O1)

Это 444 строки, которые своими руками создают, правят и закрывают задачи в репозитории. Ошибка портит настоящие данные, и ни один из шести наборов тестов её не поймает. Один обрыв сети или лимит GitHub на третьем шаге — и в списке задач появятся повторы, среди которых не разобрать настоящие.

Предлагаем: Написать тесты на синхронизацию с GitHub с подставным GitHub (разбор аргументов, поиск маркеров задач, оба генератора текста) и первым делом закрыть найденный случай: при обрыве посреди создания шагов плана соответствие «шаг → подкарточка» теряется, и повтор даст дубли.

R14 Настройки этого проекта дважды подключают хук памяти, один раз — по адресу с домашнего Linux

На маке при каждом сообщении запускается хук по несуществующему пути и завершается ошибкой; на домашнем Linux память проекта вставляется в каждый запрос дважды — экономия объёма чата, ради которой переписывали подкладку памяти, здесь не работает.

Предлагаем: Убрать из настроек проекта блок ручной регистрации хука целиком — тот же скрипт уже подключает сам плагин.

R13 Проверено, что включаются, только 10 навыков из 35 (→ O2)

Навык включается по фразе владельца. Не совпало — плагин молча работает «как обычно», без своего процесса, и владелец об этом не узнаёт. Про 25 навыков сейчас никто не знает, срабатывают ли они; ни одного сохранённого прогона нет.

Предлагаем: Дописать короткие фразы-проверки хотя бы для ключевых непокрытых: разбор идеи, сохранение памяти, завершение ветки, диагностика проблемы; результаты прогонов сохранять в репозитории.

R8 Руководство пользователя отстало на две версии и ведёт на чужой адрес (→ O2)

Это единственный подробный документ «как этим пользоваться». Любой, кому дадут плагин, получит по нему ошибку установки и команды, которых больше нет. Текущая задача правит в нём только имя пятой фазы — остальное останется.

Предлагаем: Обновить руководство: версию в заголовке, команду установки (сейчас клонирует чужой репозиторий), описания трёх удалённых команд и папку тупиков, которой в таком виде нет.

R9 Человек, зашедший на страницу плагина, не поймёт ни что это, ни как поставить (→ O2)

Корневой README — семь строк служебного блока с застывшим статусом, а именно его первым показывает страница репозитория (это и есть адрес из описания плагина). Единственная картинка конвейера в описании плагина — схема четырёх фаз без навигатора, разбора идеи и гайда.

Предлагаем: Написать README в корне: что плагин делает, как подключить, что нужно на машине; в описании плагина заменить ссылку на «историческую схему на 4 фазы» из папки черновиков на актуальную схему из семи фаз.

R17 В репозитории 27 давно влитых веток

Плагин обещает убирать за собой, а в его собственном репозитории ветки с апреля.

Предлагаем: Удалить влитые ветки (27 локальных и три на GitHub) и проверить, почему шаг уборки в навыке завершения не отрабатывает.

R18 Полоска с текущей фазой у владельца не включена

Плагин умеет показывать, на каком шаге он находится (включая «Фаза 5: Гайд по проекту»), но у владельца этой полоски нет вовсе — настройка требует вписать путь руками.

Предлагаем: Либо подключить её в личные настройки владельца, либо убрать обещание из описания плагина.

R19 Нет быстрой проверки, что плагин собран правильно

Съедет одна строка в заголовке навыка — Claude Code просто не зарегистрирует его без предупреждения. Встроенная проверка Claude Code есть и сегодня проходит, но в тесты она не включена, а именно такой тест поймал бы пустой адрес скриптов (R16) и упоминания команды, которой больше нет (так было с гайдом до переименования).

Предлагаем: Дымовой тест в общем запускаторе: у каждого навыка есть заголовок с именем и описанием, все пути из настроек существуют, описания плагина читаются; подключить к нему встроенную команду проверки плагина Claude Code.

R23 Ни одна версия плагина не отмечена меткой — откатиться не к чему

За 129 изменений вышло семь крупных версий, но ни одна не отмечена меткой или релизом на GitHub. Если после очередного обновления плагин «сломался», откатиться на известно-рабочую версию не к чему; номер версии дублируется в четырёх файлах и держится только на ручной синхронности.

Предлагаем: В шаг подъёма версии добавить создание метки версии (git-тега) и её отправку на GitHub вместе с веткой; задним числом поставить метку v7.7.0 на текущий выпуск.

R25 Память проекта называет оценку качества работающей — а это пустой каркас

Клод при загрузке памяти считает, что оценка качества уже есть, и не предложит её собрать; на вопрос «что работает» владелец получает неправду, а урок про фазы записан со ссылкой на анализ, которого не было.

Предлагаем: Перенести строку про оценку качества в памяти проекта из «работает» в честное «каркас, ни одного прогона» и поправить источник урока, который ссылается на анализ пустой таблицы.

R26 Шпаргалки по частям плагина отстали на два месяца, а Клоду велено читать их раньше кода

По инструкции Клод сначала смотрит в шпаргалку, а там нет ни отчёта, ни гайда, ни половины новых частей — он ищет вслепую или делает выводы по устаревшей картине, и каждая задача начинается с лишнего расследования.

Предлагаем: После слияния гайда пересобрать шпаргалки по частям плагина командой обновления документации, чтобы в них появились фаза 5, новые навыки и тесты; либо снять правило «сначала шпаргалка, потом код», пока они не обновляются сами.

R27 Описание проекта и реестр решений расходятся с кодом: 33 навыка вместо 35, две версии конвейера

Описание проекта — первое, что Клод читает в каждой сессии, а реестр решений — источник «почему так устроено». Когда там одновременно «4 фазы» и «7 фаз», он может объяснить владельцу не то устройство; неверные цифры подрывают доверие к остальному.

Предлагаем: Исправить число навыков в описании проекта (или убрать число вовсе, чтобы не устаревало) и пометить в реестре решений запись о четырёхфазном конвейере как заменённую семифазным.

R28 Меню для Linux предлагает команды, которых в плагине нет

Мышкой из этого меню владелец скопирует команду, которой нет, и получит ошибку вместо шага конвейера.

Предлагаем: Обновить список команд в версии меню для Linux по образцу маковской: убрать три давно удалённые команды, добавить фазы 0, 1.5 и 5.

~~**R12** Кнопочное меню в трее не знает про новую фазу и про сохранение памяти~~

Отложено — 2: Карта файлов в памяти считает содержимое папок неточно; Кусок работы над треем лежит в «кармане» git.

07 План

Сейчас · эта неделя

- Влить гайд в основную ветку и выложить 7.8.0 (R22, задача project-guide)
- Адрес скриптов в навыках — чтобы память и гайд обновлялись не на догадке (R16)
- Связь с GitHub на маке: флаг, цели, шапка README (R1, R2, R3) — или ответ по O1

Дальше · 2–4 недели

- Ответ по O2 — личный инструмент или продукт для других (открывает R8, R9, R13, R24)
- Что нужно на компьютере и проверка при старте сессии (R24)
- Автозапуск тестов, старые наборы на маке, дымовой тест сборки (R11, R15, R19)
- Тесты на синхронизацию с GitHub — если O1 = «оставить» (R10)
- Настройки проекта и проверка навыков по фразам (R14, R13)

Позже

- Ответы по O3–O6: оценка качества, навигатор, задача о доверии к памяти, черновики
- Метки версий, чистка веток, полоска фазы (R23, R17, R18)
- Порядок в памяти проекта: шпаргалки, честный статус, реестр решений (R25, R26, R27)
- Меню для Linux и отложенное: карта файлов, правки трея (R28, R20, R21)

08 Словарик

Плагин

Надстройка над программой Claude Code, которая задаёт Клоду правила работы и добавляет команды.

Навык (скилл)

Инструкция для Клода на конкретную ситуацию; включается сама по вашим словам или командой.

Хук

Маленький скрипт, который срабатывает автоматически в момент события: старт сессии, ваше сообщение, запуск команды.

Фаза (шаг конвейера)

Один из семи этапов пути от сырой идеи до готового кода; у каждого свой вход, выход и файл-результат.

Контрольная точка (чекпоинт)

Место в плане, где Клод останавливается и показывает результат, прежде чем идти дальше.

Субагент (помощник)

Отдельный экземпляр Клода, запущенный для одной подзадачи параллельно с основным разговором.

Git, ветка и мерж

Git — хранилище кода с историей изменений; ветка — отдельная копия кода под одну задачу; мерж («мержим») — вливание готовой работы обратно в основную версию.

GitHub: репозиторий, README, Issue

Сайт, где лежит копия кода проекта (репозиторий) с первой страницей описания (README) и карточками-задачами (Issue), куда плагин может зеркалить задачи и их шаги.

---

forge-plugin · гайд v1.0 от 2026-09-06 · Собрано из: код проекта, память .forge, журнал 04.06–06.09.2026 · обновлён 2026-10-23: 2 изменения после сборки · гайд устарел на 4 задачи